

Lab3 Sorting Hat

Lily Qin

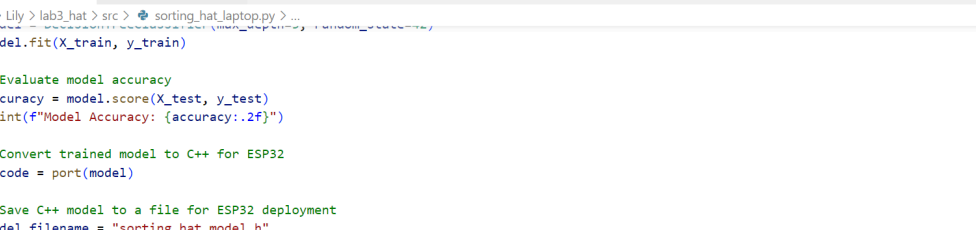
1. Create your dataset to train the sorting hat.

Create at least five responses to all questions and label them.

Talk to other students, and collect responses generated by at least five other students. This should give you at least 30 entries in the dataset. You are encouraged to collect more data as you can.

Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	House
1	1	1	1	1	1	1	1	1	1	1 Gryffindor
2	2	2	2	2	2	2	2	2	2	2 Hufflepuff
3	3	3	3	3	3	3	3	3	3	3 Ravenclaw
4	4	4	4	4	4	4	4	4	4	4 Slytherin
1	2	3	4	1	2	3	4	1	1	1 Gryffindor
2	1	4	3	3	4	2	1	4	2	2 Hufflepuff

3. Use `src/sorting_hat_laptop.py` to train your decision tree and convert it to C++ for ESP32. Depending on your dataset size, you can tune the tree depth to prevent overfitting.



The screenshot displays the Visual Studio Code interface. The editor window shows a Python file named `sorting_hat_laptop.py` with the following code:

```
19 model.fit(X_train, y_train)
20
21 # Evaluate model accuracy
22 accuracy = model.score(X_test, y_test)
23 print(f"Model Accuracy: {accuracy:.2f}")
24
25 # Convert trained model to C++ for ESP32
26 c_code = port(model)
27
28 # Save C++ model to a file for ESP32 deployment
29 model_filename = "sorting_hat_model.h"
30 with open(model_filename, "w") as f:
31     f.write(c_code)
32
33 print(f"Model successfully converted and saved as {model_filename}!")
34
```

The terminal window at the bottom shows the command execution and its output:

```
PS C:\Users\Lily> & C:/Users/Lily/.venv/Scripts/python.exe c:/Users/Lily/lab3_hat/src/sorting_hat_laptop.py
Model Accuracy: 0.50
Model successfully converted and saved as sorting_hat_model.h
PS C:\Users\Lily>
```

5. Complete the `checkButtons()` logic. The starter code includes a function stub for checking button presses and recording responses.

```
void checkButtons() {  
    bool buttonPressed = false;  
  
    while (!buttonPressed) {  
        if (digitalRead(BUTTON_A) == LOW) {  
            responses[questionIndex] = 1;  
            buttonPressed = true;  
            delay(2000);  
        }  
  
        else if (digitalRead(BUTTON_B) == LOW) {  
            responses[questionIndex] = 2;  
            buttonPressed = true;  
            delay(2000);  
        }  
  
        else if (digitalRead(BUTTON_C) == LOW) {  
            responses[questionIndex] = 3;  
            buttonPressed = true;  
            delay(2000);  
        }  
  
        else if (digitalRead(BUTTON_D) == LOW) {  
            responses[questionIndex] = 4;  
            buttonPressed = true;  
            delay(2000);  
        }  
  
        delay(500);  
    }  
  
    nextQuestion(); // Move to the next question  
}
```

 Update

Discussion

- Play with your sorting hat. Are all 10 questions important to create the sorting hat? If you were to remove some questions to improve user experience, which questions would you remove and justify your answer.

I think I will remove some questions because answering 10 questions takes a little long time. I will remove Question 6: "What to do with a mystery book?" because I don't think it can help us understand the house core.

- If you were to improve the sorting hat, what technical improvements would you make? Consider:
 - How could you improve the model's accuracy or efficiency?

In order to improve the model's accuracy and efficiency, I can ask candidates to describe their own personality with a small story, and use LLM to analyse candidates' answers. So, candidates do not need to answer ten questions. Also, we can make some words weigh and use the words that candidates may say to indicate different houses.

- What additional sensors or hardware could enhance the user experience?

We can use voice sensors to collect users' answers instead of four buttons. In this way, users can interact with the sorting hat easier.

- Does decision tree remain suitable for your choice of new sensors? If yes, carefully justify your answer. If not, what ML model would you use and explain why.

The decision tree may not be suitable when I use a new sensor. I may use Random Forest for this new solution since it is more accuracy by averaging multiple decision trees and also reduces the overfitting problem of a single decision tree.